

COM-600 · Microservicios · Gestión 2026

Práctica 1 — MongoDB CRUD (Parte B)

Estudiante:

Valencia Medina Freddy Daniel

Materia:

COM-600 Microservicios (Bloque B: Ejercicios Resolutivos B1 – B32)

■ Consultas (B1–B18)

B1. Productos de categoría 2 o 7 con nombre y precio (sin _id)

Enunciado: *Mostrá los productos de la categoría 2 o 7, con el nombre y el precio únicamente, sin el _id.*

Comando / Consulta:

```
> db.productos.find({ categoria: { $in: [2, 7] } }, { nombre: 1, precio: 1, _id: 0 })
```

```
tienda> db.productos.find({ categoria: { $in: [2, 7] } }, { nombre: 1, precio: 1, _id: 0 })

[
  { nombre: 'Singani de Tarija 750 ml', precio: 95 },
  { nombre: 'Historia de Potosí (tapa dura)', precio: 120 }
]

tienda> █
```

Explicación: Se utiliza el operador `\$in` sobre el campo `categoria` para filtrar los productos con categoría 2 o 7, y una proyección `{ nombre: 1, precio: 1, \_id: 0 }` para devolver exclusivamente los campos solicitados omitiendo explícitamente el `\_id`.

B2. Productos con precio entre 100 y 300 (inclusivo)

Enunciado: *Mostrá los productos cuyo precio esté entre 100 y 300, incluidos los dos extremos.*

Comando / Consulta:

```
> db.productos.find({ precio: { $gte: 100, $lte: 300 } })
```

```
tienda> db.productos.find({ precio: { $gte: 100, $lte: 300 } })
```

```
[
  {
    _id: ObjectId('6a7fa9d11b7c954c22369124'),
    codigo: 'CER-007',
    nombre: 'Vasija de cerámica Tiwanaku',
    precio: 260,
    stock: 7,
    stock_minimo: '5',
    activo: false,
    categoria: 5,
    categorias: [ 5, 8 ],
    etiquetas: [ 'ceramica', 'replica' ],
    medidas: { alto: 35, ancho: 28, unidad: 'cm' },
    inventario: [ { almacen: 'La Paz', cantidad: 7 } ],
    registrado: ISODate('2023-09-14T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369125'),
    codigo: 'JOY-008',
    nombre: 'Aretes de plata 950',
    precio: 190,
    stock: 25,
    stock_minimo: 8,
    activo: true,
    categoria: 6,
    categorias: [ 6 ],
    etiquetas: [ 'plata', 'potosi', 'joyeria' ],
    medidas: { alto: 4, ancho: 2, unidad: 'cm' },
    inventario: [ { almacen: 'Potosí', cantidad: 25 } ],
    variantes: [
      { nombre: 'Colgante', color: 'plata' },
      { nombre: 'Argolla', color: 'plata' }
    ],
    registrado: ISODate('2025-01-08T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369126'),
    codigo: 'LIB-009',
    nombre: 'Historia de Potosí (tapa dura)',
    precio: 120,
    stock: 26,
    stock_minimo: '10',
    activo: true,
    categoria: 7,
    categorias: [ 7 ],
    etiquetas: [ 'libro', 'historia' ],
    descuento: 20,
    medidas: { alto: 24, ancho: 17, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 20 },
      { almacen: 'Potosí', cantidad: 6 }
    ],
    registrado: ISODate('2024-08-22T00:00:00.000Z')
  }
]
```

```
tienda> █
```

Explicación: Se combinan los operadores de comparación `\$gte` (mayor o igual a 100) y `\$lte` (menor o igual a 300) dentro de la misma condición del campo `precio`.

B3. Productos que no están activos

Enunciado: *Mostrá los productos que no están activos.*

Comando / Consulta:

```
> db.productos.find({ activo: false })
```

```
tienda> db.productos.find({ activo: false })

[
  {
    _id: ObjectId('6a7fa9d11b7c954c22369124'),
    codigo: 'CER-007',
    nombre: 'Vasija de cerámica Tiwanaku',
    precio: 260,
    stock: 7,
    stock_minimo: '5',
    activo: false,
    categoria: 5,
    categorias: [ 5, 8 ],
    etiquetas: [ 'ceramica', 'replica' ],
    medidas: { alto: 35, ancho: 28, unidad: 'cm' },
    inventario: [ { almacen: 'La Paz', cantidad: 7 } ],
    registrado: ISODate('2023-09-14T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369129'),
    codigo: 'TEX-012',
    nombre: 'Poncho de vicuña',
    stock: 2,
    stock_minimo: 1,
    activo: false,
    categoria: 4,
    categorias: [ 4, 8 ],
    etiquetas: [ 'textil', 'vicuna', 'lujo' ],
    medidas: { alto: 150, ancho: 140, unidad: 'cm' },
    inventario: [ { almacen: 'La Paz', cantidad: 2 } ],
    registrado: ISODate('2025-07-01T00:00:00.000Z')
  }
]

tienda> █
```

Explicación: Se filtra por la condición booleana `{ activo: false }`, devolviendo los dos productos inactivos de la tienda (CER-007 y TEX-012).

B4. Productos cuyo nombre empieza con la letra A o con la letra C

Enunciado: *Mostrá los productos cuyo nombre empieza con la letra A o con la letra C.*

Comando / Consulta:

```
> db.productos.find({ nombre: /^[AC]/ })
```

```
tienda> db.productos.find({ nombre: /^[AC]/ })
```

```
[
  {
    _id: ObjectId('6a7fa9d11b7c954c2236911e'),
    codigo: 'ART-001',
    nombre: 'Charango de madera de naranjo',
    precio: 850,
    stock: 12,
    stock_minimo: 3,
    activo: true,
    categoria: 3,
    categorias: [ 3, 8 ],
    etiquetas: [ 'musica', 'artesanía', 'madera' ],
    medidas: { alto: 60, ancho: 20, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 5 },
      { almacen: 'La Paz', cantidad: 7 }
    ],
    registrado: ISODate('2024-03-12T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c2236911f'),
    codigo: 'ART-002',
    nombre: 'Aguayo tejido a mano',
    precio: 320,
    stock: 40,
    stock_minimo: 10,
    activo: true,
    categoria: 4,
    categorias: [ 4, 8 ],
    etiquetas: [ 'textil', 'artesanía', 'tradicional' ],
    descuento: 10,
    medidas: { alto: 120, ancho: 100, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 25 },
      { almacen: 'Potosí', cantidad: 15 }
    ],
    registrado: ISODate('2024-01-20T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369122'),
    codigo: 'ALM-005',
    nombre: 'Chocolate de Sucre 100 g',
    precio: 28,
    stock: 150,
    stock_minimo: 40,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'cacao', 'sucre', 'dulce' ],
    descuento: 15,
    medidas: { alto: 12, ancho: 7, unidad: 'cm' },
    inventario: [ { almacen: 'Sucre', cantidad: 150 } ],
    registrado: ISODate('2025-04-18T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369123'),
    codigo: 'TEX-006',
    nombre: 'Chompa de alpaca',
    precio: 480,
```

Explicación: Se utiliza una expresión regular con anclaje al inicio `^[AC]/``, permitiendo evaluar eficientemente si el nombre del producto comienza con 'A' o con 'C'.

B5. Productos que tienen el campo variantes

Enunciado: *Mostrá los productos que tienen el campo variantes.*

Comando / Consulta:

```
> db.productos.find({ variantes: { $exists: true } })
```

```
tienda> db.productos.find({ variantes: { $exists: true } })
```

```
[
  {
    _id: ObjectId('6a7fa9d11b7c954c22369123'),
    codigo: 'TEX-006',
    nombre: 'Chompa de alpaca',
    precio: 480,
    stock: 18,
    stock_minimo: 5,
    activo: true,
    categoria: 4,
    categorias: [ 4, 8 ],
    etiquetas: [ 'textil', 'alpaca', 'abrigo' ],
    medidas: { alto: 70, ancho: 55, unidad: 'cm' },
    inventario: [
      { almacen: 'La Paz', cantidad: 10 },
      { almacen: 'Sucre', cantidad: 8 }
    ],
    variantes: [
      { nombre: 'Talla M', color: 'gris' },
      { nombre: 'Talla L', color: 'negro' }
    ],
    registrado: ISODate('2024-06-30T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369125'),
    codigo: 'JOY-008',
    nombre: 'Aretes de plata 950',
    precio: 190,
    stock: 25,
    stock_minimo: 8,
    activo: true,
    categoria: 6,
    categorias: [ 6 ],
    etiquetas: [ 'plata', 'potosi', 'joyeria' ],
    medidas: { alto: 4, ancho: 2, unidad: 'cm' },
    inventario: [ { almacen: 'Potosí', cantidad: 25 } ],
    variantes: [
      { nombre: 'Colgante', color: 'plata' },
      { nombre: 'Argolla', color: 'plata' }
    ],
    registrado: ISODate('2025-01-08T00:00:00.000Z')
  }
]
```

```
tienda> █
```

Explicación: El operador `\$exists: true` comprueba si el campo `variantes` se encuentra definido en el documento, sin importar su contenido ni tipo.

B6. Productos donde stock_minimo se cargó como texto en vez de número

Enunciado: *Encontrá los productos donde stock_minimo se cargó como texto en vez de número.*

Comando / Consulta:


```
> db.productos.find({ stock_minimo: { $type: "string" } })
```

```
tienda> db.productos.find({ stock_minimo: { $type: "string" } })

[
  {
    _id: ObjectId('6a7fa9d11b7c954c22369124'),
    codigo: 'CER-007',
    nombre: 'Vasija de cerámica Tiwanaku',
    precio: 260,
    stock: 7,
    stock_minimo: '5',
    activo: false,
    categoria: 5,
    categorias: [ 5, 8 ],
    etiquetas: [ 'ceramica', 'replica' ],
    medidas: { alto: 35, ancho: 28, unidad: 'cm' },
    inventario: [ { almacen: 'La Paz', cantidad: 7 } ],
    registrado: ISODate('2023-09-14T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369126'),
    codigo: 'LIB-009',
    nombre: 'Historia de Potosí (tapa dura)',
    precio: 120,
    stock: 26,
    stock_minimo: '10',
    activo: true,
    categoria: 7,
    categorias: [ 7 ],
    etiquetas: [ 'libro', 'historia' ],
    descuento: 20,
    medidas: { alto: 24, ancho: 17, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 20 },
      { almacen: 'Potosí', cantidad: 6 }
    ],
    registrado: ISODate('2024-08-22T00:00:00.000Z')
  }
]

tienda> █
```

Explicación: El operador `$type: "string"` permite auditar y detectar registros que contienen inconsistencias de tipado BSON (valores de stock mínimo almacenados como cadenas de texto '5' y '10').

B7. Los 4 productos con más stock (solo nombre y stock)

Enunciado: *Mostrá los 4 productos con más stock, con el nombre y el stock únicamente.*

Comando / Consulta:

```
> db.productos.find({}, { nombre: 1, stock: 1, _id: 0 }).sort({ stock: -1 }).limit(4)
```



```
tienda> db.productos.find({}, { nombre: 1, stock: 1, _id: 0 }).sort({ stock: -1 }).limit(4)
```

```
[  
  { nombre: 'Quinoa real orgánica 1 kg', stock: 200 },  
  { nombre: 'Chocolate de Sucre 100 g', stock: 150 },  
  { nombre: 'Café de los Yungas 250 g', stock: 90 },  
  { nombre: 'Singani de Tarija 750 ml', stock: 60 }  
]
```

```
tienda> █
```

Explicación: Se aplica una proyección `{ nombre: 1, stock: 1, \_id: 0 }`, se ordena de mayor a menor stock con `.sort({ stock: -1 })` y se acota el cursor a los primeros 4 resultados con `.limit(4)`.

B8. Segunda página de listado ordenado por nombre ascendente (de 4 en 4)

Enunciado: *Mostrar la segunda página de un listado ordenado por nombre ascendente, de 4 en 4.*

Comando / Consulta:

```
> db.productos.find().sort({ nombre: 1 }).skip(4).limit(4)
```

```
tienda> db.productos.find().sort({ nombre: 1 }).skip(4).limit(4)
```

```
[
  {
    _id: ObjectId('6a7fa9d11b7c954c22369122'),
    codigo: 'ALM-005',
    nombre: 'Chocolate de Sucre 100 g',
    precio: 28,
    stock: 150,
    stock_minimo: 40,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'cacao', 'sucre', 'dulce' ],
    descuento: 15,
    medidas: { alto: 12, ancho: 7, unidad: 'cm' },
    inventario: [ { almacen: 'Sucre', cantidad: 150 } ],
    registrado: ISODate('2025-04-18T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369123'),
    codigo: 'TEX-006',
    nombre: 'Chompa de alpaca',
    precio: 480,
    stock: 18,
    stock_minimo: 5,
    activo: true,
    categoria: 4,
    categorias: [ 4, 8 ],
    etiquetas: [ 'textil', 'alpaca', 'abrigo' ],
    medidas: { alto: 70, ancho: 55, unidad: 'cm' },
    inventario: [
      { almacen: 'La Paz', cantidad: 10 },
      { almacen: 'Sucre', cantidad: 8 }
    ],
    variantes: [
      { nombre: 'Talla M', color: 'gris' },
      { nombre: 'Talla L', color: 'negro' }
    ],
    registrado: ISODate('2024-06-30T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369126'),
    codigo: 'LIB-009',
    nombre: 'Historia de Potosí (tapa dura)',
    precio: 120,
    stock: 26,
    stock_minimo: '10',
    activo: true,
    categoria: 7,
    categorias: [ 7 ],
    etiquetas: [ 'libro', 'historia' ],
    descuento: 20,
    medidas: { alto: 24, ancho: 17, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 20 },
      { almacen: 'Potosí', cantidad: 6 }
    ],
    registrado: ISODate('2024-08-22T00:00:00.000Z')
  },
  {

```

Explicación: Para paginar con tamaño de página 4, se ordenan los productos alfabéticamente con ``sort({ nombre: 1 })``, se omiten los 4 elementos de la primera página con ``skip(4)`` y se obtienen los siguientes 4 con ``limit(4)``.

B9. Productos con la etiqueta "organico" o "artesanía"

Enunciado: *Mostrá los productos que tengan la etiqueta "organico" o la etiqueta "artesanía".*

Comando / Consulta:

```
> db.productos.find({ etiquetas: { $in: ["organico", "artesanía"] } })
```

```
tienda> db.productos.find({ etiquetas: { $in: ["organico", "artesanía"] } })
```

```
[
  {
    _id: ObjectId('6a7fa9d11b7c954c2236911e'),
    codigo: 'ART-001',
    nombre: 'Charango de madera de naranjo',
    precio: 850,
    stock: 12,
    stock_minimo: 3,
    activo: true,
    categoria: 3,
    categorias: [ 3, 8 ],
    etiquetas: [ 'musica', 'artesanía', 'madera' ],
    medidas: { alto: 60, ancho: 20, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 5 },
      { almacen: 'La Paz', cantidad: 7 }
    ],
    registrado: ISODate('2024-03-12T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c2236911f'),
    codigo: 'ART-002',
    nombre: 'Aguayo tejido a mano',
    precio: 320,
    stock: 40,
    stock_minimo: 10,
    activo: true,
    categoria: 4,
    categorias: [ 4, 8 ],
    etiquetas: [ 'textil', 'artesanía', 'tradicional' ],
    descuento: 10,
    medidas: { alto: 120, ancho: 100, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 25 },
      { almacen: 'Potosí', cantidad: 15 }
    ],
    registrado: ISODate('2024-01-20T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369120'),
    codigo: 'ALM-003',
    nombre: 'Quinoa real orgánica 1 kg',
    precio: 45,
    stock: 200,
    stock_minimo: 50,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'organico', 'grano', 'altiplano' ],
    medidas: { alto: 25, ancho: 15, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 120 },
      { almacen: 'Cochabamba', cantidad: 80 }
    ],
    registrado: ISODate('2025-02-05T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369127'),
    codigo: 'ART-010',

```

Explicación: El operador `\$in` sobre un campo de tipo array (`etiquetas`) verifica si al menos uno de los elementos coincide con cualquiera de los valores especificados, sin necesidad de usar `\$or`.

B10. Productos cuyo array categorias tiene exactamente un elemento

Enunciado: *Mostrá los productos cuyo array categorias tenga exactamente un elemento.*

Comando / Consulta:

```
> db.productos.find({ categorias: { $size: 1 } })
```

```
tienda> db.productos.find({ categorias: { $size: 1 } })
```

```
[
  {
    _id: ObjectId('6a7fa9d11b7c954c22369120'),
    codigo: 'ALM-003',
    nombre: 'Quinua real orgánica 1 kg',
    precio: 45,
    stock: 200,
    stock_minimo: 50,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'organico', 'grano', 'altiplano' ],
    medidas: { alto: 25, ancho: 15, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 120 },
      { almacen: 'Cochabamba', cantidad: 80 }
    ],
    registrado: ISODate('2025-02-05T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369121'),
    codigo: 'BEB-004',
    nombre: 'Singani de Tarija 750 ml',
    precio: 95,
    stock: 60,
    stock_minimo: 15,
    activo: true,
    categoria: 2,
    categorias: [ 2 ],
    etiquetas: [ 'destilado', 'tarija' ],
    medidas: { alto: 30, ancho: 8, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 35 },
      { almacen: 'Tarija', cantidad: 25 }
    ],
    registrado: ISODate('2024-11-11T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369122'),
    codigo: 'ALM-005',
    nombre: 'Chocolate de Sucre 100 g',
    precio: 28,
    stock: 150,
    stock_minimo: 40,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'cacao', 'sucre', 'dulce' ],
    descuento: 15,
    medidas: { alto: 12, ancho: 7, unidad: 'cm' },
    inventario: [ { almacen: 'Sucre', cantidad: 150 } ],
    registrado: ISODate('2025-04-18T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369125'),
    codigo: 'JOY-008',
    nombre: 'Aretes de plata 950',
    precio: 190,
    stock: 25,
  }
]
```

Explicación: El operador ``$size: 1`` filtra aquellos documentos donde la cantidad total de elementos dentro del array ``categorias`` es exactamente igual a 1.

B11. Productos con menos de 10 unidades en La Paz (\$elemMatch vs sin \$elemMatch)

Enunciado: *Mostrá los productos con menos de 10 unidades en el almacén de La Paz. Escribí las dos versiones —sin `$elemMatch` y con `$elemMatch`— y anotá cuántos documentos devuelve cada una y por qué son distintas.*

Comando / Consulta:

```
> db.productos.find({ "inventario.almacen": "La Paz", "inventario.cantidad": { $lt: 10 } })  
> db.productos.find({ inventario: { $elemMatch: { almacen: "La Paz", cantidad: { $lt: 10 } } } })
```



```
tienda> db.productos.find({ "inventario.almacen": "La Paz", "inventario.cantidad": { $lt: 10 } })
```

```
[
  {
    _id: ObjectId('6a7fa9d11b7c954c2236911e'),
    codigo: 'ART-001',
    nombre: 'Charango de madera de naranjo',
    precio: 850,
    stock: 12,
    stock_minimo: 3,
    activo: true,
    categoria: 3,
    categorias: [ 3, 8 ],
    etiquetas: [ 'musica', 'artesanía', 'madera' ],
    medidas: { alto: 60, ancho: 20, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 5 },
      { almacen: 'La Paz', cantidad: 7 }
    ],
    registrado: ISODate('2024-03-12T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369123'),
    codigo: 'TEX-006',
    nombre: 'Chompa de alpaca',
    precio: 480,
    stock: 18,
    stock_minimo: 5,
    activo: true,
    categoria: 4,
    categorias: [ 4, 8 ],
    etiquetas: [ 'textil', 'alpaca', 'abrigo' ],
    medidas: { alto: 70, ancho: 55, unidad: 'cm' },
    inventario: [
      { almacen: 'La Paz', cantidad: 10 },
      { almacen: 'Sucre', cantidad: 8 }
    ],
    variantes: [
      { nombre: 'Talla M', color: 'gris' },
      { nombre: 'Talla L', color: 'negro' }
    ],
    registrado: ISODate('2024-06-30T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369124'),
    codigo: 'CER-007',
    nombre: 'Vasija de cerámica Tiwanaku',
    precio: 260,
    stock: 7,
    stock_minimo: '5',
    activo: false,
    categoria: 5,
    categorias: [ 5, 8 ],
    etiquetas: [ 'cerámica', 'replica' ],
    medidas: { alto: 35, ancho: 28, unidad: 'cm' },
    inventario: [ { almacen: 'La Paz', cantidad: 7 } ],
    registrado: ISODate('2023-09-14T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369129'),
    codigo: 'TEX-012',

```

Justificación técnica / Explicación: • Sin \$elemMatch (devuelve 4 documentos): Evalúa los criterios de forma independiente sobre el array `inventario`. Se cuela erróneamente el producto TEX-006 (Chompa de alpaca), ya que cumple `almacen: "La Paz"` en un elemento (cantidad: 10) y `cantidad < 10` en otro elemento diferente (almacén: "Sucre", cantidad: 8).

• Con \$elemMatch (devuelve 3 documentos): Exige de forma estricta que ambas condiciones (`almacen == "La Paz"` y `cantidad < 10`) se cumplan dentro del MISMO subdocumento del array, retornando con precisión únicamente los productos correctos: ART-001, CER-007 y TEX-012.

B12. Productos cuya primera categoría es 1

Enunciado: *Mostrá los productos cuya primera categoría del array categorías sea 1.*

Comando / Consulta:

```
> db.productos.find({ "categorias.0": 1 })
```

```

tienda> db.productos.find({ "categorias.0": 1 })

[
  {
    _id: ObjectId('6a7fa9d11b7c954c22369120'),
    codigo: 'ALM-003',
    nombre: 'Quinoa real orgánica 1 kg',
    precio: 45,
    stock: 200,
    stock_minimo: 50,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'organico', 'grano', 'altiplano' ],
    medidas: { alto: 25, ancho: 15, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 120 },
      { almacen: 'Cochabamba', cantidad: 80 }
    ],
    registrado: ISODate('2025-02-05T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369122'),
    codigo: 'ALM-005',
    nombre: 'Chocolate de Sucre 100 g',
    precio: 28,
    stock: 150,
    stock_minimo: 40,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'cacao', 'sucre', 'dulce' ],
    descuento: 15,
    medidas: { alto: 12, ancho: 7, unidad: 'cm' },
    inventario: [ { almacen: 'Sucre', cantidad: 150 } ],
    registrado: ISODate('2025-04-18T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369128'),
    codigo: 'ALM-011',
    nombre: 'Café de los Yungas 250 g',
    precio: 60,
    stock: 90,
    stock_minimo: 20,
    activo: true,
    categoria: 1,
    categorias: [ 1, 2 ],
    etiquetas: [ 'cafe', 'yungas', 'organico' ],
    medidas: { alto: 18, ancho: 10, unidad: 'cm' },
    inventario: [
      { almacen: 'La Paz', cantidad: 50 },
      { almacen: 'Sucre', cantidad: 40 }
    ],
    registrado: ISODate('2025-05-10T00:00:00.000Z')
  }
]

```

```
tienda> █
```

Explicación: Mediante la notación de punto con posición indexada (`"categorias.0"`), se consulta específicamente el primer elemento (posición 0) del array `categorias`.

B13. Productos registrados durante el año 2025

Enunciado: *Mostrá los productos registrados durante el año 2025.*

Comando / Consulta:

```
> db.productos.find({ registrado: { $gte: ISODate("2025-01-01"), $lt: ISODate("2026-01-01") } })
```

```
tienda> db.productos.find({ registrado: { $gte: ISODate("2025-01-01"), $lt: ISODate("2026-01-01")
} })
```

```
[
  {
    _id: ObjectId('6a7fa9d11b7c954c22369120'),
    codigo: 'ALM-003',
    nombre: 'Quinua real orgánica 1 kg',
    precio: 45,
    stock: 200,
    stock_minimo: 50,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'organico', 'grano', 'altiplano' ],
    medidas: { alto: 25, ancho: 15, unidad: 'cm' },
    inventario: [
      { almacen: 'Sucre', cantidad: 120 },
      { almacen: 'Cochabamba', cantidad: 80 }
    ],
    registrado: ISODate('2025-02-05T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369122'),
    codigo: 'ALM-005',
    nombre: 'Chocolate de Sucre 100 g',
    precio: 28,
    stock: 150,
    stock_minimo: 40,
    activo: true,
    categoria: 1,
    categorias: [ 1 ],
    etiquetas: [ 'cacao', 'sucre', 'dulce' ],
    descuento: 15,
    medidas: { alto: 12, ancho: 7, unidad: 'cm' },
    inventario: [ { almacen: 'Sucre', cantidad: 150 } ],
    registrado: ISODate('2025-04-18T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369125'),
    codigo: 'JOY-008',
    nombre: 'Aretes de plata 950',
    precio: 190,
    stock: 25,
    stock_minimo: 8,
    activo: true,
    categoria: 6,
    categorias: [ 6 ],
    etiquetas: [ 'plata', 'potosi', 'joyeria' ],
    medidas: { alto: 4, ancho: 2, unidad: 'cm' },
    inventario: [ { almacen: 'Potosí', cantidad: 25 } ],
    variantes: [
      { nombre: 'Colgante', color: 'plata' },
      { nombre: 'Argolla', color: 'plata' }
    ],
    registrado: ISODate('2025-01-08T00:00:00.000Z')
  },
  {
    _id: ObjectId('6a7fa9d11b7c954c22369127'),
    codigo: 'ART-010',
    nombre: 'Máscara de diablada',

```

Explicación: Para delimitar el año 2025 completo, se utiliza un rango de tipo `ISODate`: mayor o igual al 1 de enero de 2025 (`\$gte`) y estrictamente menor al 1 de enero de 2026 (`\$lt`).

B14. Cantidad de productos activos

Enunciado: *Averiguá cuántos productos están activos. Se pide el número, no la lista.*

Comando / Consulta:

```
> db.productos.countDocuments({ activo: true })
```

```
tienda> db.productos.countDocuments({ activo: true })

10

tienda> █
```

Explicación: El método `countDocuments({ activo: true })` realiza el conteo directamente en la base de datos y retorna el valor escalar (10) sin transferir la lista de documentos.

B15. Pedidos de Sucre con total mayor a 300

Enunciado: *Mostrá los pedidos de la ciudad de Sucre cuyo total sea mayor a 300.*

Comando / Consulta:

```
> db.pedidos.find({ ciudad: "Sucre", total: { $gt: 300 } })
```

```
tienda> db.pedidos.find({ ciudad: "Sucre", total: { $gt: 300 } })

[
  {
    _id: 1,
    cliente: 'Ana Vargas',
    ciudad: 'Sucre',
    estado: 'pendiente',
    items: [ { codigo: 'ART-001', cantidad: 1, precio: 850 } ],
    total: 850,
    fecha: ISODate('2025-06-01T14:00:00.000Z')
  }
]

tienda> █
```

Explicación: Se consultan los pedidos combinando `ciudad: "Sucre"` y `total: { \$gt: 300 }` en un AND implícito dentro del objeto de filtro.

B16. Pedidos que incluyan el producto "ALM-005"

Enunciado: *Mostrá los pedidos que incluyan el producto de código "ALM-005".*

Comando / Consulta:

```
> db.pedidos.find({ "items.codigo": "ALM-005" })
```

```
tienda> db.pedidos.find({ "items.codigo": "ALM-005" })

[
  {
    _id: 4,
    cliente: 'Ana Vargas',
    ciudad: 'Sucre',
    estado: 'pendiente',
    items: [ { codigo: 'ALM-005', cantidad: 10, precio: 28 } ],
    total: 280,
    fecha: ISODate('2025-06-09T11:15:00.000Z')
  }
]

tienda> █
```

Explicación: Con la notación de punto `"items.codigo"`, MongoDB busca dentro del array de subdocumentos `items` aquellos pedidos que contienen dicho código de producto.

B17. Pedidos que tengan más de un ítem

Enunciado: *Mostrá los pedidos que tengan más de un ítem.*

Comando / Consulta:

```
> db.pedidos.find({ "items.1": { $exists: true } })
```



```
tienda> db.pedidos.find({ "items.1": { $exists: true } })
```

```
[
  {
    _id: 2,
    cliente: 'Luis Mamani',
    ciudad: 'La Paz',
    estado: 'enviado',
    items: [
      { codigo: 'ALM-003', cantidad: 4, precio: 45 },
      { codigo: 'ALM-011', cantidad: 2, precio: 60 }
    ],
    total: 300,
    fecha: ISODate('2025-06-03T09:30:00.000Z')
  },
  {
    _id: 6,
    cliente: 'Rosa Quispe',
    ciudad: 'Potosí',
    estado: 'enviado',
    items: [
      { codigo: 'ART-002', cantidad: 3, precio: 320 },
      { codigo: 'LIB-009', cantidad: 1, precio: 120 }
    ],
    total: 1080,
    fecha: ISODate('2025-06-14T13:20:00.000Z')
  }
]
```

```
tienda> █
```

Explicación: Dado que MongoDB no tiene un operador de longitud mayor que sobre arrays, se comprueba la existencia del índice 1 ("items.1": { \$exists: true }), lo que garantiza que el array posee al menos dos elementos.

B18. Lista de clientes distintos que hicieron pedidos

Enunciado: *Mostrá la lista de clientes distintos que hicieron pedidos.*

Comando / Consulta:

```
> db.pedidos.distinct("cliente")
```

```
tienda> db.pedidos.distinct("cliente")
```

```
[ 'Ana Vargas', 'Carlos Nina', 'Luis Mamani', 'Rosa Quispe' ]
```

```
tienda> █
```


Explicación: El método `distinct("cliente")` extrae los valores únicos del campo `cliente` de toda la colección `pedidos`, eliminando nombres duplicados.

■ Creación (B19-B22)

B19. Inserción de un producto nuevo completo

Enunciado: *Inserta un producto nuevo que tenga, como mínimo: un array de textos, un subdocumento y un array de subdocumentos. Los datos son de tu invención, pero tienen que ser coherentes con los demás productos.*

Comando / Consulta:

```
> db.productos.insertOne({
  codigo: "TEX-013",
  nombre: "Gorro de lana de llama",
  precio: 75,
  stock: 30,
  stock_minimo: 5,
  activo: true,
  categoria: 4,
  categorias: [4, 8],
  etiquetas: ["textil", "lana", "artesanía"],
  medidas: {
    alto: 25,
    ancho: 20,
    unidad: "cm"
  },
  inventario: [
    { almacen: "Sucre", cantidad: 20 },
    { almacen: "La Paz", cantidad: 10 }
  ],
  registrado: new Date("2025-08-01T00:00:00Z")
})
```

```

tienda> db.productos.insertOne({
  codigo: "TEX-013",
  nombre: "Gorro de lana de llama",
  precio: 75,
  stock: 30,
  stock_minimo: 5,
  activo: true,
  categoria: 4,
  categorias: [4, 8],
  etiquetas: ["textil", "lana", "artesanía"],
  medidas: {
    alto: 25,
    ancho: 20,
    unidad: "cm"
  },
  inventario: [
    { almacen: "Sucre", cantidad: 20 },
    { almacen: "La Paz", cantidad: 10 }
  ],
  registrado: new Date("2025-08-01T00:00:00Z")
})

{
  acknowledged: true,
  insertedId: ObjectId('6a7fa9ebbf6417fb595249be')
}

tienda> █

```

Explicación: Se ejecuta `insertOne()` para insertar un nuevo documento que cumple con las 3 condiciones de estructura: array de textos (`etiquetas`), subdocumento (`medidas`) y array de subdocumentos (`inventario`).

B20. Inserción de tres productos en una sola instrucción

Enunciado: *Inserta tres productos más en una sola instrucción.*

Comando / Consulta:

```

> db.productos.insertMany([
  {
    codigo: "CER-014",
    nombre: "Taza de arcilla esmaltada",
    precio: 35,
    stock: 50,
    activo: true,
    categoria: 5,
    etiquetas: ["cerámica", "artesanía"]
  },
  {
    codigo: "BEB-015",
    nombre: "Vino tinto de Cinti 750 ml",
    precio: 80,

```

```
    stock: 45,  
    activo: true,  
    categoria: 2,  
    etiquetas: ["bebida", "vino", "cinti"]  
  },  
  {  
    codigo: "JOY-016",  
    nombre: "Anillo de filigrana de plata",  
    precio: 220,  
    stock: 15,  
    activo: true,  
    categoria: 6,  
    etiquetas: ["joyeria", "plata"]  
  }  
])
```

```

tienda> db.productos.insertMany([
  {
    codigo: "CER-014",
    nombre: "Taza de arcilla esmaltada",
    precio: 35,
    stock: 50,
    activo: true,
    categoria: 5,
    etiquetas: ["ceramica", "artesanía"]
  },
  {
    codigo: "BEB-015",
    nombre: "Vino tinto de Cinti 750 ml",
    precio: 80,
    stock: 45,
    activo: true,
    categoria: 2,
    etiquetas: ["bebida", "vino", "cinti"]
  },
  {
    codigo: "JOY-016",
    nombre: "Anillo de filigrana de plata",
    precio: 220,
    stock: 15,
    activo: true,
    categoria: 6,
    etiquetas: ["joyeria", "plata"]
  }
])

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a7fa9edfd962fbbfadf24bc'),
    '1': ObjectId('6a7fa9edfd962fbbfadf24bd'),
    '2': ObjectId('6a7fa9edfd962fbbfadf24be')
  }
}

tienda> █

```

Explicación: El método `insertMany()` inserta un arreglo de documentos JSON en una sola operación atómica en el servidor.

B21. Inserción de un pedido con `_id` 7 y dos ítems

Enunciado: Inserta un pedido con `_id` 7, de un cliente que no exista todavía, con dos ítems.

Comando / Consulta:

```

> db.pedidos.insertOne({
  _id: 7,
  cliente: "Mateo Fernandez",
  ciudad: "Cochabamba",
  estado: "pendiente",

```

```

items: [
  { codigo: "ART-001", cantidad: 2, precio: 850 },
  { codigo: "ALM-005", cantidad: 5, precio: 28 }
],
total: 1840,
fecha: new Date("2025-07-15T10:00:00Z")
})

```

```

tienda> db.pedidos.insertOne({
  _id: 7,
  cliente: "Mateo Fernandez",
  ciudad: "Cochabamba",
  estado: "pendiente",
  items: [
    { codigo: "ART-001", cantidad: 2, precio: 850 },
    { codigo: "ALM-005", cantidad: 5, precio: 28 }
  ],
  total: 1840,
  fecha: new Date("2025-07-15T10:00:00Z")
})

{ acknowledged: true, insertedId: 7 }

tienda> █

```

Explicación: Se registra un nuevo pedido en la colección `pedidos` con `\_id: 7`, nuevo cliente y 2 ítems con su importe total calculado ($2 \times 850 + 5 \times 28 = 1840$).

B22. Inserción sin precio y conteo de productos sin precio

Enunciado: Inserta un producto sin el campo precio. Después contá cuántos productos no tienen precio y explicá el número que sale.

Comando / Consulta:

```

> db.productos.insertOne({
  codigo: "ESC-017",
  nombre: "Escultura tallada en madera",
  stock: 4,
  stock_minimo: 1,
  activo: true,
  categoria: 3,
  etiquetas: ["madera", "escultura"]
})
> db.productos.countDocuments({ precio: { $exists: false } })

```

```

tienda> db.productos.insertOne({
  codigo: "ESC-017",
  nombre: "Escultura tallada en madera",
  stock: 4,
  stock_minimo: 1,
  activo: true,
  categoria: 3,
  etiquetas: ["madera", "escultura"]
})

{
  acknowledged: true,
  insertedId: ObjectId('6a7fa9f02e1376640b3a99f8')
}

tienda> db.productos.countDocuments({ precio: { $exists: false } })

2

tienda> █

```

Justificación técnica / Explicación: El resultado del conteo es 2. Esto se debe a que la base de datos ya contenía inicialmente un producto sin el campo `precio` (TEX-012, Poncho de vicuña, como parte de las irregularidades intencionales de la práctica). Al insertar este nuevo producto (ESC-017) omitiendo también el campo `precio`, la consulta `{ precio: { \$exists: false } }` detecta ambos documentos (1 original + 1 nuevo = 2).

■ Actualización (B23-B28)

B23. Subir 10% el precio en categoría 4 y análisis de TEX-012 (\$mul)

Enunciado: *Subí un 10 % el precio de los productos de la categoría 4. Después mirá el precio del poncho de vicuña (TEX-012) y explicá qué le pasó y por qué.*

Comando / Consulta:

```

> load("seed.js")
> db.productos.updateMany({ categoria: 4 }, { $mul: { precio: 1.1 } })
> db.productos.find({ codigo: "TEX-012" }, { codigo: 1, nombre: 1, precio: 1, _id: 0 })

```

```

tienda> load("seed.js")

productos: 12 y pedidos: 6
true

tienda> db.productos.updateMany({ categoria: 4 }, { $mul: { precio: 1.1 } })

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}

tienda> db.productos.find({ codigo: "TEX-012" }, { codigo: 1, nombre: 1, precio: 1, _id: 0 })

[ { codigo: 'TEX-012', nombre: 'Poncho de vicuña', precio: 0 } ]

tienda> █

```

Justificación técnica / Explicación: El poncho de vicuña (TEX-012) pertenece a la categoría 4 pero originalmente no tenía el campo `precio`. En MongoDB, cuando el operador `\$mul` se ejecuta sobre un campo que NO existe en el documento, MongoDB no genera error ni ignora el registro; crea automáticamente el campo numérico y lo inicializa en 0 ($0 * 1.1 = 0$). Por lo tanto, el poncho queda con `precio: 0`.

B24. Actualizar pedidos enviados a entregados con fecha del servidor

Enunciado: *Pasá a "entregado" todos los pedidos que estén en "enviado", y dejales registrada la fecha de entrega con la hora del servidor. Una sola instrucción.*

Comando / Consulta:

```

> db.pedidos.updateMany(
  { estado: "enviado" },
  { $set: { estado: "entregado" }, $currentDate: { fecha_entrega: true } }
)

```

```

tienda> db.pedidos.updateMany(
  { estado: "enviado" },
  { $set: { estado: "entregado" }, $currentDate: { fecha_entrega: true } }
)

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}

tienda> █

```

Explicación: Se utiliza `updateMany()` combinando `\$set` para modificar el estado y `\$currentDate: { fecha\_entrega: true }` para registrar la fecha/hora actual del sistema.

B25. Agregar etiqueta "liquidacion" a productos inactivos sin duplicar

Enunciado: Agregá la etiqueta "liquidacion" a todos los productos inactivos, de manera que no se duplique si alguno ya la tuviera.

Comando / Consulta:

```

> db.productos.updateMany(
  { activo: false },
  { $addToSet: { etiquetas: "liquidacion" } }
)

```

```

tienda> db.productos.updateMany(
  { activo: false },
  { $addToSet: { etiquetas: "liquidacion" } }
)

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}

tienda> █

```

Explicación: El operador `\$addToSet` agrega el valor "liquidacion" al array `etiquetas` únicamente si no se encuentra presente, evitando duplicados.

B26. Eliminar campo stock_minimo cargado como texto (\$unset)

Enunciado: Borrá el campo stock_minimo únicamente en los productos donde está cargado como texto. Verificá después que no queda ninguno así.

Comando / Consulta:

```
> db.productos.updateMany(
  { stock_minimo: { $type: "string" } },
  { $unset: { stock_minimo: "" } }
)
> db.productos.find({ stock_minimo: { $type: "string" } })
```

```
tienda> db.productos.updateMany(
  { stock_minimo: { $type: "string" } },
  { $unset: { stock_minimo: "" } }
)

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}

tienda> db.productos.find({ stock_minimo: { $type: "string" } })

// (sin salida / cursor vacío)

tienda> █
```

Explicación: Se aplica `\$unset: { stock\_minimo: "" }` a los documentos donde `{ stock\_minimo: { \$type: "string" } }` para remover el campo. La verificación posterior con `find()` devuelve un cursor vacío (0 documentos).

B27. Agregar nuevo almacén al Café de los Yungas (\$push)

Enunciado: Agregá al café de los Yungas (ALM-011) un almacén nuevo: "Camiri" con 5 unidades, sin tocar los almacenes que ya tiene.

Comando / Consulta:

```
> db.productos.updateOne(
  { codigo: "ALM-011" },
  { $push: { inventario: { almacen: "Camiri", cantidad: 5 } } }
)
```

```

tienda> db.productos.updateOne(
  { codigo: "ALM-011" },
  { $push: { inventario: { almacen: "Camiri", cantidad: 5 } } }
)

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

tienda> █

```

Explicación: El operador `\$push` añade el nuevo subdocumento `{ almacen: "Camiri", cantidad: 5 }` al final del array `inventario` sin modificar los almacenes existentes (La Paz y Sucre).

B28. Actualizar o crear producto BEB-030 con upsert

Enunciado: Con una sola instrucción, actualizá el producto de código "BEB-030" si existe y créalo si no existe, con un nombre, precio 40 y stock 25.

Comando / Consulta:

```

> db.productos.updateOne(
  { codigo: "BEB-030" },
  { $set: { nombre: "Cerveza artesanal de trigo", precio: 40, stock: 25 } },
  { upsert: true }
)

```

```

tienda> db.productos.updateOne(
  { codigo: "BEB-030" },
  { $set: { nombre: "Cerveza artesanal de trigo", precio: 40, stock: 25 } },
  { upsert: true }
)

{
  acknowledged: true,
  insertedId: ObjectId('6a7fa9fa5978df25c26d74b1'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}

tienda> █

```

Explicación: Con `{ upsert: true }`, si el producto no existe, MongoDB lo crea automáticamente combinando los datos del filtro `{ codigo: "BEB-030" }` con los valores provistos en `\$set`.

■ Eliminación (B29–B32)

B29. Contar y eliminar pedidos cancelados

Enunciado: *Contá cuántos pedidos están cancelados y recién después bórralos. Entregá los dos comandos, en ese orden.*

Comando / Consulta:

```
> db.pedidos.countDocuments({ estado: "cancelado" })
> db.pedidos.deleteMany({ estado: "cancelado" })
```

```
tienda> db.pedidos.countDocuments({ estado: "cancelado" })

1

tienda> db.pedidos.deleteMany({ estado: "cancelado" })

{ acknowledged: true, deletedCount: 1 }

tienda> █
```

Explicación: Se aplica la buena práctica de verificar primero la cantidad de registros a eliminar con `countDocuments()` (1 pedido cancelado) antes de invocar `deleteMany()`.

B30. Eliminar un solo producto con etiqueta "textil"

Enunciado: *Eliminá un solo producto que tenga la etiqueta "textil". Ojo: hay más de uno, y el comando tiene que borrar exactamente uno.*

Comando / Consulta:

```
> db.productos.deleteOne({ etiquetas: "textil" })
```

```
tienda> db.productos.deleteOne({ etiquetas: "textil" })

{ acknowledged: true, deletedCount: 1 }

tienda> █
```

Explicación: El método `deleteOne()` elimina únicamente el primer documento coincidente en el orden de almacenamiento que contiene la etiqueta "textil".

B31. Eliminar productos con stock menor a 5

Enunciado: *Eliminó los productos con stock menor a 5.*

Comando / Consulta:

```
> db.productos.deleteMany({ stock: { $lt: 5 } })
```



```
tienda> db.productos.deleteMany({ stock: { $lt: 5 } })  
  
{ acknowledged: true, deletedCount: 2 }  
  
tienda> █
```

Explicación: El método `deleteMany()` elimina de forma masiva todos los documentos cuyo campo `stock` es estrictamente menor a 5.

B32. Restaurar base de datos y verificar conteos

Enunciado: *Restauró la base al punto de partida y verificó que quedaron 12 productos y 6 pedidos.*

Comando / Consulta:

```
> load("seed.js")  
> db.productos.countDocuments()  
> db.pedidos.countDocuments()
```

```
tienda> load("seed.js")

productos: 12 y pedidos: 6
true

tienda> db.productos.countDocuments()

12

tienda> db.pedidos.countDocuments()

6

tienda> █
```

Explicación: Se ejecuta `load("seed.js")` para repoblar la base de datos con los datos originales, verificando con `countDocuments()` que quedan exactamente 12 productos y 6 pedidos.